

Useful links:

<http://sqlfiddle.com/>

<http://sqlfiddle.com/>

- <http://sqlfiddle.com/#!4/b81c3/6> Kurzor nefunguje

- [illegible]

where podil < 100)
group by cisloA,jmenoA;

Jednodussi varianta:n

```
SELECT cisloA, jmenoA  
FROM Autor NATURAL JOIN Je_autorem  
GROUP BY cisloA, jmenoA  
HAVING AVG(podil)=100;
```

c) Napište příkaz pro vytvoření databázového kurzoru, který umožní zpřístupnit řádky výsledku dotazu: Na kolika publikacích se podíleli jednotliví autoři z ústavu 'UITS'? (cisloA, jmenoA, pocet_publicaci) Dále uveďte, zda by bylo možné tento kurzor použít pro aktualizaci záznamů databáze a své tvrzení zdůvodněte. [4 body]

```
DECLARE cursor1 CURSOR FOR  
SELECT cisloA, jmenoA , COUNT(*) pocet_publicaci  
FROM Autor NATURAL JOIN Je_autorem  
WHERE ustav= 'UITS'  
GROUP BY cisloA , jmenoA
```

Podle mě spíš:

```
CURSOR cursor1 IS  
SELECT cisloA,jmenoA,COUNT(*) as pocet_publicaci  
FROM Je_autorem  
WHERE ustav = 'UITS'  
GROUP BY cisloA,jmenoA;
```

Nemůže se použít na aktualizaci, protože obsahuje prvek **GROUP BY** a skládá se z více než jedné tabulky.

- 2.) Predpokladajte časť relačnej databázy leteckej spoločnosti, ktorá obsahuje tabuľky Cestujuci (cisloC , meno, mesto), Let (cisloL , odkial, kam, datum, cas) a Letenka (cisloC, cisloL ,miesto, trieda). U cestujúceho je uvedené jeho jednoznačné číslo, meno a mesto bydliska, pri lete je uvedené jednoznačné číslo, odkiaľ a kam sa letí a dátum a čas odletu. V tabuľke Letenka je číslo cestujúceho a číslo letu, ktorého sa cestujúci účastní, ďalej miesto v lietadle a trieda. Primárne kľúče tabuliek sú podčiarknuté. Napíšte v jazyku SQL nasledujúce príkazy:

<http://sqlfiddle.com/#!4/d63cb0> // Nevyplnená tabuľka pre tabuľku Letenka a chýbajúci údaj "datum" tak som to dorobil v ďalšom linku:

<http://sqlfiddle.com/#!4/105bcc/7>

<http://sqlfiddle.com/#!4/177e4/1> //pridal som tam aj komentý pre SELECTy

Napíšte dotaz: Do ktorých destinácií bolo vystavených viac leteniek než do Madridu? (kam, pocet_leteniek)

```
SELECT kam, COUNT(*) AS pocet_leteniek
FROM Let Lt JOIN Letenka La ON Lt.cisloL = La.cisloL
GROUP BY kam
WHERE kam = 'Madrid'
//rôzne varianty zápisu spojenia
//použitie kam <> 'Madrid' v dotaze vyššej úrovne nie je chyba
^^ bullshit :) po GROUP BY nemôže byť WHERE a navíc to vybere pouze lety z madridu
```

Původně tu bylo:

```
SELECT kam, COUNT(*) AS pocet_leteniek
FROM Let Lt JOIN Letenka La ON Lt.cisloL = La.cisloL
GROUP BY kam
HAVING COUNT(*) > (
    SELECT COUNT(*)
    FROM Let Lt JOIN Letenka La ON Lt.cisloL = La.cisloL
    WHERE kam = 'Madrid')
```

Lahšia varianta (imho):

```
SELECT kam, COUNT(*) AS pocet_leteniek
FROM Let NATURAL JOIN Letenka
GROUP BY kam
HAVING COUNT(*) > (SELECT COUNT(*)
    FROM Let NATURAL JOIN Letenka
    WHERE kam = 'Madrid')
```

Napíšte príkaz pre vytvorenie databázového pohľadu obsahujúceho informácie o cestujúcich, ktorí v roku 2018 aspoň raz leteli z Prahy (cisloC, meno, mesto). Ďalej uveďte, aký význam by malo pridanie klauzule WITH CHECK OPTION k definícii tohto pohľadu a tvrdenie zdôvodnite.

```
CREATE VIEW pohlad AS
SELECT cisloC, meno, mesto
FROM Cestujuci C JOIN Letenka La ON C.cisloC = La.CisloC JOIN Let Lt
ON La.cisloL = Lt.cisloL
WHERE odkial='Praha' AND datum BETWEEN '2018-01-01' AND '2018-12-31'
```

Lahšia varianta (imho):

```
CREATE VIEW pohlad AS
SELECT cisloC, meno, mesto
FROM Cestujuci NATURAL JOIN Letenka NATURAL JOIN Let
WHERE odkial = 'Praha' AND datum BETWEEN '2018-01-01' AND '2018-12-31'
```

WITH CHECK OPTION údajne podľa DC delá: zabraňuje aby sa dali upravovať riadky pred pohľad, ktoré ten pohľad nezobrazuje

Predpokladajte, že v tabuľke Let je pridaný atribút pocet_cestujucich . Napíšte príkaz pre vytvorenie databázového triggera , ktorý v prípade vloženia nového záznamu do tabuľky Letenka zvýši o jedničku hodnotu atribútu pocet_cestujucich .

```
CREATE TRIGGER trigger1
AFTER INSERT ON Letenka
FOR EACH ROW
BEGIN
    UPDATE Let
    SET pocet_cestujucich = pocet_cestujucich + 1
    WHERE cisloL = :NEW.cisloL; //keď sa vyskytne nový záznam
END;
//možné použiť aj BEFORE
```

Napíšte dotaz: Ktorí cestujúci lietajú len z Prahy do Londýna? (cisloC, meno)

```
SELECT cisloC, meno
FROM Cestujuci C JOIN Letenka La ON C.cisloC = La.cisloC JOIN Let Lt
ON La.cisloL = Lt.cisloL
```

```

WHERE odkial = 'Praha' AND kam = 'Londyn' AND NOT EXISTS (
    SELECT * FROM Letenka La JOIN Let Lt ON La.cisloL = Lt.cisloL
    WHERE C.cisloC = La.cisloC AND (odkial<>'Praha' OR kam<>'Londyn')

```

//moje riešenie (odskúšané na SQL fiddle (treba si doplnit trasu Praha, Londýn))

```

SELECT cisloC, meno
FROM Cestujuci NATURAL JOIN Letenka NATURAL JOIN Let
WHERE odkial = 'Praha' AND kam = 'Londýn' AND cisloC NOT IN(
    SELECT cisloC
    FROM Cestujuci NATURAL JOIN Letenka NATURAL JOIN Let
    WHERE odkial <> 'Praha' OR kam <> 'Londýn')

```

Napíšte príkaz pre vytvorenie databázového pohľadu obsahujúceho informácie o počtoch .predaných leteniek do business triedy pre jednotlivé linky (odkial, kam, pocet_leteniek). Linka zahrňuje všetky lety medzi dvoma rovnakými letiskami (napr. Praha-Londýn). Ďalej uveďte, aký význam by malo pridanie klauzule WITH CHECK OPTION k definícii tohto pohľadu a tvrdenie odôvodnite

```

CREATE VIEW pohlad AS
SELECT odkial, kam, COUNT (*) AS pocet_leteniek
FROM Let Lt JOIN Letenka La ON Lt.cisloL = La.cisloL
WHERE trieda = 'business'
GROUP BY odkial, kam
//rôzne varianty zápisu spojenia

```

Predpokladajte, že v tabuľke Let je pridaný atribút pocet_volnych_miest . Napíšte príkaz pre vytvorenie databázového triggera, ktorý v prípade zmazania záznamu z tabuľky Letenka zvýši o jedničku hodnotu atribútu pocet_volnych_miest .

```

CREATE TRIGGER trigger1
AFTER DELETE ON Letenka
FOR EACH ROW
BEGIN
    UPDATE Let
    SET pocet_volnych_miest += 1
    WHERE cisloL = :OLD.cisloL;
END;

```

//možné použiť aj BEFORE

3) Uvažujte časť relačnej databázy banky týkajúcej sa klientov a účtov, ktorá obsahuje tabuľky Klient (RC , meno, mesto, rok_nar), Disponuje (RC, c_uctu , limit) a Ucet (c_uctu , stav, typ, vlastnik). O klientovi ukladáme jeho rodné číslo, meno, mesto bydliska a rok narodenia, o účte jednoznačné číslo účtu, stav, typ (napr. sporiaci, bežný, atď.) a cudzí kľúč do tabuľky Klient odkazujúci sa na vlastníka. V tabuľke Disponuje sú údaje o ďalších disponentoch (mimo vlastníka) jednotlivých účtov (cudzí kľúče do tabuliek Klient a Ucet) a ich limity výberu.

Primárne

kľúče sú podčiarknuté. Zapište v SQL nasledujúce dotazy/príkazy (zoznam stĺpcov v zátvorke udáva požadovaný tvar výsledkov):

<http://sqlfiddle.com/#!4/200bb9>

<http://sqlfiddle.com/#!4/200bb9/214> //komments added

Ktoré účty, na ktorých je stav vyšší než 100 000 Kč, nemajú žiadneho disponenta?

(c_uctu, stav, typ)

```
SELECT c_uctu, stav, typ
FROM Ucet NATURAL LEFT JOIN Disponuje
WHERE stav > 100000 AND RC IS NULL
```

Další varianta:

```
select c_uctu, stav, typ
from Ucet
where stav > 100000 and c_uctu not in (select c_uctu from Ucet natural join Disponuje)
```

Ktorí disponenti majú na všetkých účtoch, s ktorými disponujú, limit nižší než 5000 Kč?

(RC, meno, mesto)

```
SELECT RC, meno, mesto
FROM Klient K, Disponuje D
WHERE K.RC = D.RC AND limit < 5000
AND NOT EXISTS
    (SELECT * FROM Disponuje D2
     WHERE K.RC = D2.RC
     AND limit >= 5000)
```

Nebo:

```
select RC, meno, mesto
from Klient natural join Disponuje
group by RC,meno,mesto
having MAX(limitUcet) < 5000;
```

```
//overeno , funguje
SELECT DISTINCT RC, meno, mesto
FROM Klient NATURAL JOIN Disponuje
WHERE limitUcet < 5000 AND RC NOT IN (
    SELECT RC FROM Disponuje
    WHERE limitUcet >=5000)
```

Napište príkaz pre pridanie nového atribútu datum_zmeny do tabuľky Klient . Ďalej napíšte príkaz pre vytvorenie databázového triggera, ktorý v prípade vloženia záznamu alebo jeho modifikácie uloží do atribútu datum_zmeny aktuálny dátum:

```
ALTER TABLE Klient ADD datum_zmeny DATE
CREATE TRIGGER set_datum
BEFORE INSERT OR UPDATE ON Klient
FOR EACH ROW
BEGIN
    :NEW.datum_zmeny := SYSDATE;
END
```

Ktorí klienti disponujú s účtami 10 a viac rôznych vlastníkov? (RC, meno, mesto, pocet_uctov)

```
SELECT RC, meno, mesto, COUNT (*) pocet_uctov
FROM Klient NATURAL JOIN Disponuje NATURAL JOIN Ucet
GROUP BY RC, meno, mesto
HAVING COUNT (DISTINCT vlastnik) >= 10
```

S ktorými účtami disponuje najvyšší počet disponentov? Uvažujte prípad, že môže byť viacero účtov s rovnakým maximálnym počtom disponentov (c_uctu, typ, pocet_disponentov)

```
SELECT c_uctu, typ, COUNT (*) pocet_disponentov
FROM Ucet NATURAL JOIN Disponuje
GROUP BY c_uctu, typ
HAVING COUNT (*) >= ALL (
    SELECT COUNT(*)
    FROM Ucet NATURAL JOIN Disponuje
```

GROUP BY c_uctu, typ)

Napište príkaz pre pridanie nového atribútu datum_zmeny do tabuľky Ucet . Ďalej napíšte príkaz pre vytvorenie databázového triggera, ktorý v prípade vloženia záznamu alebo jeho modifikácie uloží do atribútu datum_zmeny aktuálny dátum:

```
ALTER TABLE Ucet ADD datum_zmeny DATE
CREATE TRIGGER set_datum
BEFORE INSERT OR UPDATE ON Ucet
FOR EACH ROW
BEGIN
    :NEW.datum_zmeny := SYSDATE;
END
```